# Министерство науки и высшего образования РФ

Отчёт по практикуму

## Проектирование и администрирование базы данных учёта экзаменационной сессии

по дисциплине «Системы управления базами данных»

Вариант 7

Выполнил: Лузан Р. Д.

2026 г.

# Содержание

**Введение** **2**

**1 Задание 1.1. Проектирование схемы** **2**
1.1 Постановка . . . 2
1.2 Назначение базы . . . 2
1.3 Сущности . . . 2

**2 Задание 1.2. Создание и заполнение таблиц** **3**
2.1 Постановка . . . 3
2.2 DDL . . . 3
2.3 Данные . . . 4

**3 Задание 1.3. Операторы манипулирования** **6**
3.1 Постановка . . . 6
3.2 Выборка 1. Нагрузка преподавателя . . . 6
3.3 Выборка 2. Дисциплины без сессии . . . 7
3.4 Выборка 3. Рейтинг групп . . . 7
3.5 Выборка 4. Ближайший контроль . . . 8
3.6 Изменение 1. Перевод четвёртого курса . . . 8
3.7 Изменение 2. Дифференцированный зачёт . . . 9
3.8 Вставка 1. Новая дисциплина и зачёт . . . 9
3.9 Вставка 2. Пересдача баз данных . . . 10
3.10 Удаление 1. Снятие пересдачи . . . 10
3.11 Удаление 2. Дисциплины вне сессии . . . 11

**4 Задание 1.4. Контроль целостности** **11**
4.1 Постановка . . . 11
4.2 Сводка . . . 11
4.3 Потерянное обновление . . . 11
4.4 Грязное и неповторяющееся чтение . . . 12
4.5 Фантом . . . 12
4.6 Сериализация . . . 12
4.7 Триггеры . . . 12
4.8 Выводы по заданию 1.4 . . . 12

**5 Задание 2.1. Управление доступом** **13**
5.1 Постановка . . . 13
5.2 Права и представления . . . 13
5.3 Проверка от имени test . . . 13
5.4 Выводы по заданию 2.1 . . . 13

**6 Задание 2.2. Функции и PL/pgSQL** **13**
6.1 Постановка . . . 13
6.2 Назначение экзамена . . . 14
6.3 Ведомость преподавателя . . . 14
6.4 Перенос сессии группы . . . 14
6.5 Сводка факультета . . . 14
6.6 Функция против представления . . . 14
6.7 Выводы по заданию 2.2 . . . 14

**Заключение** **15**

# Введение

Цель практикума — спроектировать базу данных предметной области варианта 7, реализовать её в PostgreSQL и отработать манипулирование данными, транзакции, триггеры, роли, представления и функции PL/pgSQL.

Предметная область — учёт экзаменационной сессии: учебные группы, дисциплины и контрольные мероприятия. СУБД — PostgreSQL 16. Скрипты из каталога `sql` выполнены на этой версии, числовые результаты получены этим прогоном.

# 1 Задание 1.1. Проектирование схемы

## 1.1 Постановка

Спроектировать концептуальную схему базы данных своего варианта: сущности, связи, первичные и внешние ключи. Схема должна отвечать задаче предметной области и находиться как минимум в третьей нормальной форме. Типы столбцов на этом шаге не фиксируются.

## 1.2 Назначение базы

База ведётся деканатом. По ней отвечают на вопросы: какие группы учатся на факультете и курсе, какие дисциплины закреплены за кафедрой, когда и в какой форме группа сдаёт дисциплину и кто принимает контроль. Схема хранит состав групп, перечень дисциплин и расписание контрольных мероприятий.

## 1.3 Сущности

- **StudentGroups** — учебная группа. Суррогатный ключ `Id`. Естественный ключ `GroupCode` уникален, но в связи не входит: код группы можно переименовать, не переписывая экзамены. Курс, численность и факультет зависят только от группы.
- **Subjects** — дисциплина. Суррогатный ключ `Id`, уникальное название `SubjectName`. Кафедра и объём часов описывают дисциплину, а не конкретный экзамен.
- **ExamSessions** — ассоциативная сущность «группа сдаёт дисциплину». Ключ `Id`. На группу и дисциплину ссылаются внешние ключи `StudentGroupId` и `SubjectId`. Дата, преподаватель и форма контроля описывают конкретный контроль.

Связи — две связи «один ко многим». Экзамен и есть таблица связи между группой и дисциплиной. Транзитивных зависимостей нет: факультет не определяет курс, кафедра не определяет часы. Схема удовлетворяет третьей нормальной форме. Неключевые атрибуты зависят только от сущности, а не друг от друга.

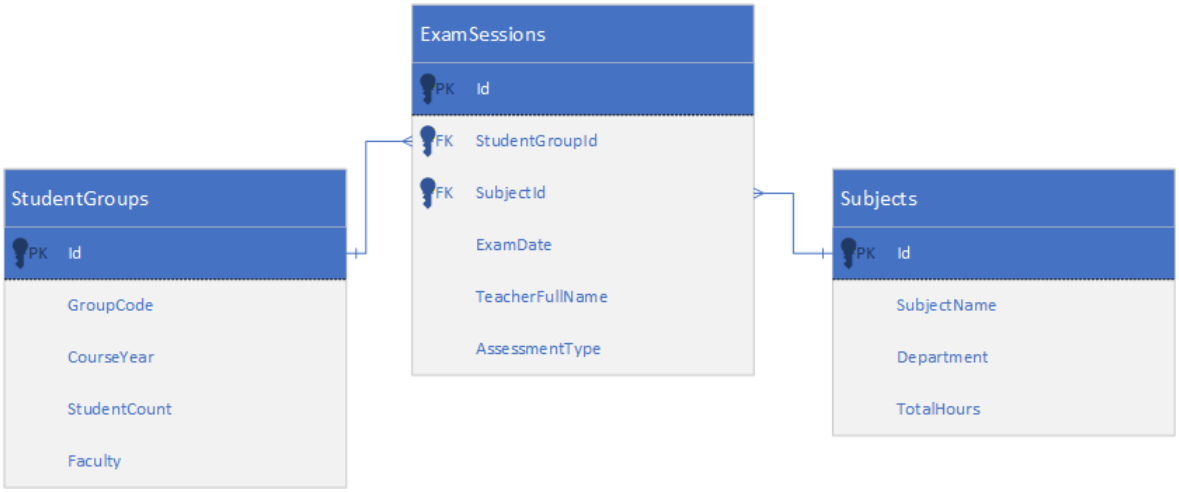



Рис. 1: ER-диаграмма варианта 7

# 2 Задание 1.2. Создание и заполнение таблиц

## 2.1 Постановка

По схеме задания 1.1 подготовить DDL с типами, ключами и декларативными ограничениями. Заполнить стержневые сущности не менее чем десятью строками, ассоциативную — не менее чем пятнадцатью. Снять SQL-дамп базы.

## 2.2 DDL

Имена столбцов записаны в PascalCase, как в выданной схеме варианта. Первичные ключи — `UUID`, значение по умолчанию даёт `gen_random_uuid()`. Проверки не дают завести курс вне диапазона 1–6, отрицательную численность, нулевой объём дисциплины и произвольную строку вместо формы контроля. Внешние ключи обновляются и удаляются каскадно. Уникальность тройки группа–дисциплина–дата задана отдельным ограничением: суррогатный ключ повторный экзамен не запрещает. Индексы по обоим внешним ключам ускоряют соединение.

```
-- Практическое задание 1.2. Вариант 7.
-- Предметная область: учёт экзаменационных сессий учебных групп.
-- СУБД: PostgreSQL 16. Суррогатные ключи UUID (gen_random_uuid).

DROP SCHEMA IF EXISTS exam CASCADE;
CREATE SCHEMA exam;
SET search_path TO exam, public;

-- Стержневая сущность: учебная группа.
CREATE TABLE "StudentGroups" (
    "Id"           UUID         DEFAULT gen_random_uuid() PRIMARY KEY,
    "GroupCode"    VARCHAR(20)  NOT NULL UNIQUE,
    "CourseYear"   SMALLINT     NOT NULL CHECK ("CourseYear" BETWEEN 1 AND 6),
    "StudentCount" INTEGER      NOT NULL CHECK ("StudentCount" >= 0),
    "Faculty"      VARCHAR(100) NOT NULL
);

-- Стержневая сущность: учебная дисциплина.
CREATE TABLE "Subjects" (
    "Id"          UUID         DEFAULT gen_random_uuid() PRIMARY KEY,
    "SubjectName" VARCHAR(150) NOT NULL UNIQUE,
```

```
    "Department"  VARCHAR(100) NOT NULL,
    "TotalHours"  INTEGER      NOT NULL CHECK ("TotalHours" > 0)
);

-- Ассоциативная сущность: экзамен группы по дисциплине в конкретную дату.
CREATE TABLE "ExamSessions" (
    "Id"               UUID         DEFAULT gen_random_uuid() PRIMARY KEY,
    "StudentGroupId"   UUID         NOT NULL,
    "SubjectId"        UUID         NOT NULL,
    "ExamDate"         DATE         NOT NULL,
    "TeacherFullName"  VARCHAR(150) NOT NULL,
    "AssessmentType"   VARCHAR(50)  NOT NULL
        CHECK ("AssessmentType" IN ('Экзамен', 'Зачёт', 'Дифференцированный зачёт')),

    CONSTRAINT "FK_ExamSessions_StudentGroups"
        FOREIGN KEY ("StudentGroupId") REFERENCES "StudentGroups" ("Id")
        ON UPDATE CASCADE ON DELETE CASCADE,

    CONSTRAINT "FK_ExamSessions_Subjects"
        FOREIGN KEY ("SubjectId") REFERENCES "Subjects" ("Id")
        ON UPDATE CASCADE ON DELETE CASCADE,

    CONSTRAINT "UQ_ExamSessions_Group_Subject_Date"
        UNIQUE ("StudentGroupId", "SubjectId", "ExamDate")
);

CREATE INDEX "IX_ExamSessions_StudentGroupId" ON "ExamSessions" ("StudentGroupId");
CREATE INDEX "IX_ExamSessions_SubjectId" ON "ExamSessions" ("SubjectId");

COMMENT ON TABLE "StudentGroups" IS 'Учебная группа: код, курс, численность, факультет';
COMMENT ON TABLE "Subjects" IS 'Дисциплина: название, кафедра, объём в часах';
COMMENT ON TABLE "ExamSessions" IS 'Экзамен: группа, дисциплина, дата, преподаватель, форма
    контроля';
```

## 2.3 Данные

Загружены 12 групп, 13 дисциплин и 22 экзамена. Экзамен вставляется не по `UUID`, а соединением с группами и дисциплинами по коду и названию, поэтому скрипт не зависит от сгенерированных ключей. Дисциплина «Философия» загружена без экзамена: по ней строится выборка нераспределённых дисциплин и последующее удаление. Полный скрипт вставки:

```
-- Заполнение таблиц. Вариант 7.
-- Стержневые сущности: не менее 10 строк. Ассоциативная: не менее 15 строк.
-- Связи экзамена идут по суррогатным ключам, естественные ключи остаются UNIQUE.
SET search_path TO exam, public;

INSERT INTO "StudentGroups" ("GroupCode", "CourseYear", "StudentCount", "Faculty") VALUES
    ('ИВТ-21', 4, 24, 'Институт компьютерных наук'),
    ('ИВТ-22', 3, 26, 'Институт компьютерных наук'),
    ('ИВТ-23', 2, 28, 'Институт компьютерных наук'),
    ('ПИ-21',  4, 22, 'Институт компьютерных наук'),
    ('ПИ-22',  3, 25, 'Институт компьютерных наук'),
    ('ПМ-21',  4, 20, 'Институт прикладной математики'),
    ('ПМ-22',  3, 21, 'Институт прикладной математики'),
    ('ЭК-21',  4, 30, 'Инженерно-экономический институт'),
    ('ЭК-22',  3, 27, 'Инженерно-экономический институт'),
    ('МТ-21',  4, 18, 'Механико-технологический институт'),
    ('МТ-22',  3, 19, 'Механико-технологический институт'),
```

```
    ('ИБ-23',  2, 23, 'Институт␣компьютерных␣наук');

INSERT INTO "Subjects" ("SubjectName", "Department", "TotalHours") VALUES
    ('Базы␣данных',                  'Кафедра␣вычислительных␣систем', 144),
    ('Операционные␣системы',         'Кафедра␣вычислительных␣систем', 108),
    ('Алгоритмы␣и␣структуры␣данных', 'Кафедра␣прикладной␣математики', 144),
    ('Математический␣анализ',        'Кафедра␣высшей␣математики',     216),
    ('Дискретная␣математика',        'Кафедра␣высшей␣математики',     144),
    ('Экономика␣предприятия',        'Кафедра␣экономики',             108),
    ('Бухгалтерский␣учёт',           'Кафедра␣экономики',              72),
    ('Технология␣машиностроения',    'Кафедра␣технологии␣машиностроения', 180),
    ('Информационная␣безопасность',  'Кафедра␣защиты␣информации',     108),
    ('Компьютерные␣сети',            'Кафедра␣вычислительных␣систем', 108),
    ('Теория␣вероятностей',          'Кафедра␣высшей␣математики',     108),
    ('Программирование',             'Кафедра␣вычислительных␣систем', 180),
    ('Философия',                    'Кафедра␣гуманитарных␣наук',      72);

INSERT INTO "ExamSessions"
    ("StudentGroupId", "SubjectId", "ExamDate", "TeacherFullName", "AssessmentType")
SELECT g."Id", s."Id", v.exam_date, v.teacher, v.kind
FROM (VALUES
    ('ИВТ-21', 'Базы␣данных',                 DATE '2026-01-12', 'Соколова␣Анна␣Петровна',
       'Экзамен'),
    ('ИВТ-22', 'Базы␣данных',                 DATE '2026-01-14', 'Соколова␣Анна␣Петровна',
       'Экзамен'),
    ('ИВТ-23', 'Базы␣данных',                 DATE '2026-06-10', 'Соколова␣Анна␣Петровна',
       'Зачёт'),
    ('ПИ-21',  'Базы␣данных',                 DATE '2026-01-16', 'Орлов␣Игорь␣Сергеевич',
       'Экзамен'),
    ('ИВТ-21', 'Операционные␣системы',        DATE '2026-01-19', 'Кузнецов␣Павел␣Андреевич'
    , 'Экзамен'),
    ('ИВТ-22', 'Операционные␣системы',        DATE '2026-01-20', 'Кузнецов␣Павел␣Андреевич'
    , 'Экзамен'),
    ('ПИ-22',  'Операционные␣системы',        DATE '2026-01-21', 'Кузнецов␣Павел␣Андреевич'
    , 'Дифференцированный␣зачёт'),
    ('ИВТ-21', 'Алгоритмы␣и␣структуры␣данных', DATE '2026-01-23', 'Морозова␣Елена␣Викторовна
    ', 'Экзамен'),
    ('ПМ-21',  'Алгоритмы␣и␣структуры␣данных', DATE '2026-01-26', 'Морозова␣Елена␣Викторовна
    ', 'Экзамен'),
    ('ПМ-22',  'Дискретная␣математика',       DATE '2026-01-15', 'Волков␣Дмитрий␣Игоревич',
       'Экзамен'),
    ('ПМ-21',  'Математический␣анализ',       DATE '2026-01-13', 'Волков␣Дмитрий␣Игоревич',
       'Экзамен'),
    ('ИВТ-23', 'Дискретная␣математика',       DATE '2026-06-08', 'Волков␣Дмитрий␣Игоревич',
       'Зачёт'),
    ('ЭК-21',  'Экономика␣предприятия',       DATE '2026-01-22', 'Белова␣Марина␣Олеговна',
       'Экзамен'),
    ('ЭК-22',  'Бухгалтерский␣учёт',          DATE '2026-01-27', 'Белова␣Марина␣Олеговна',
       'Дифференцированный␣зачёт'),
    ('ЭК-21',  'Бухгалтерский␣учёт',          DATE '2026-01-28', 'Лебедев␣Артём␣Николаевич'
    , 'Зачёт'),
    ('МТ-21',  'Технология␣машиностроения',   DATE '2026-01-29', 'Ершов␣Сергей␣Павлович',
       'Экзамен'),
    ('МТ-22',  'Технология␣машиностроения',   DATE '2026-01-30', 'Ершов␣Сергей␣Павлович',
       'Экзамен'),
    ('ИБ-23',  'Информационная␣безопасность', DATE '2026-06-12', 'Новикова␣Ольга␣Дмитриевна
    ', 'Экзамен'),
    ('ИВТ-22', 'Компьютерные␣сети',           DATE '2026-01-18', 'Орлов␣Игорь␣Сергеевич',
       'Экзамен'),
    ('ПИ-21',  'Компьютерные␣сети',           DATE '2026-01-18', 'Орлов␣Игорь␣Сергеевич',
```

```
       'Экзамен'),
    ('ПМ-21',  'Теория␣вероятностей',          DATE '2026-06-15', 'Волков␣Дмитрий␣Игоревич',
       'Экзамен'),
    ('ИВТ-23', 'Программирование',             DATE '2026-06-05', 'Морозова␣Елена␣Викторовна
    ', 'Дифференцированный␣зачёт')
) AS v(group_code, subject_name, exam_date, teacher, kind)
JOIN "StudentGroups" g ON g."GroupCode" = v.group_code
JOIN "Subjects" s ON s."SubjectName" = v.subject_name;
```

После загрузки: `StudentGroups` — 12 строк, `Subjects` — 13, `ExamSessions` — 22.

# 3 Задание 1.3. Операторы манипулирования

## 3.1 Постановка

Подготовить четыре осмысленные выборки и по два запроса на вставку, изменение и удаление. В запросах должны встретиться соединение, обобщённое табличное выражение, оконная функция, агрегирование, группировка, вложенный и коррелированный подзапросы.

## 3.2 Выборка 1. Нагрузка преподавателя

Зимняя сессия считается через CTE. К экзамену присоединяется численность группы по `StudentGroupId`, преподаватели группируются, в выборку попадают те, у кого не меньше двух контролей. Оконная функция считает долю преподавателя во всей сессии, `ROUND` оставляет три знака.

```
WITH winter AS (
    SELECT * FROM "ExamSessions"
    WHERE "ExamDate" >= DATE '2026-01-01'
      AND "ExamDate" <  DATE '2026-02-01'
)
SELECT w."TeacherFullName" AS "Преподаватель",
       COUNT(*) AS "Экзаменов",
       COUNT(DISTINCT w."StudentGroupId") AS "Групп",
       SUM(g."StudentCount") AS "Студентов",
       ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 3)
           AS "ДоляСессииПроцент"
FROM winter w
JOIN "StudentGroups" g ON g."Id" = w."StudentGroupId"
GROUP BY w."TeacherFullName"
HAVING COUNT(*) >= 2
ORDER BY "Студентов" DESC;
```

Таблица 1: Нагрузка преподавателей, январь 2026

| Преподаватель | Экзаменов | Групп | Студентов | Доля, % |
|---|---|---|---|---|
| Кузнецов П. А. | 3 | 3 | 75 | 18,750 |
| Орлов И. С. | 3 | 2 | 70 | 18,750 |
| Белова М. О. | 2 | 2 | 57 | 12,500 |
| Соколова А. П. | 2 | 2 | 50 | 12,500 |
| Морозова Е. В. | 2 | 2 | 44 | 12,500 |
| Волков Д. И. | 2 | 2 | 41 | 12,500 |
| Ершов С. П. | 2 | 2 | 37 | 12,500 |

Семь строк дают $3+3+2+2+2+2+2 = 16$ экзаменов. Доли сходятся: $100 \cdot 16/16 = 100$.

## 3.3 Выборка 2. Дисциплины без сессии

Левое соединение оставляет дисциплины, которым не нашлось экзамена. Коррелированный подзапрос считает средний объём кафедры.

```
SELECT s."SubjectName", s."Department", s."TotalHours",
       (SELECT ROUND(AVG(s2."TotalHours"), 3) FROM "Subjects" s2
        WHERE s2."Department" = s."Department") AS "СреднийЧасыКафедры"
FROM "Subjects" s
LEFT JOIN "ExamSessions" e ON e."SubjectId" = s."Id"
WHERE e."SubjectId" IS NULL;
```

Результат — одна строка: Философия, кафедра гуманитарных наук, 72 часа. Среднее по кафедре тоже 72, дисциплина на кафедре единственная.

## 3.4 Выборка 3. Рейтинг групп

CTE считает число контролей по `StudentGroupId`. `ROW_NUMBER() OVER (PARTITION BY "Faculty")` расставляет места отдельно внутри факультета. Первые места внутри факультета: ЭК-21 (2 контроля), ИВТ-21 (3), ПМ-21 (3), МТ-21 (1). Всего 12 строк, по одной на группу: левое соединение сохраняет группы без экзаменов.

```
WITH load AS (
    SELECT g."Faculty", g."GroupCode", g."CourseYear",
           COUNT(e."ExamDate") AS "ЧислоКонтролей",
           ROUND(COUNT(e."ExamDate")::numeric
               / NULLIF(g."StudentCount", 0), 3) AS "КонтролейНаСтудента"
    FROM "StudentGroups" g
    LEFT JOIN "ExamSessions" e ON e."StudentGroupId" = g."Id"
    GROUP BY g."Faculty", g."GroupCode", g."CourseYear", g."StudentCount"
)
SELECT "Faculty" AS "Факультет", "GroupCode" AS "Группа",
       "CourseYear" AS "Курс", "ЧислоКонтролей", "КонтролейНаСтудента",
       ROW_NUMBER() OVER (
           PARTITION BY "Faculty"
           ORDER BY "ЧислоКонтролей" DESC, "GroupCode"
       ) AS "МестоНаФакультете"
FROM load
ORDER BY "Факультет", "МестоНаФакультете";
```

Таблица 2: Вывод выборки 3, 12 строк

| Факультет | Группа | Курс | Контролей | На студента | Место |
|---|---|---|---|---|---|
| Инженерно-экономический институт | ЭК-21 | 4 | 2 | 0,067 | 1 |
| Инженерно-экономический институт | ЭК-22 | 3 | 1 | 0,037 | 2 |
| Институт компьютерных наук | ИВТ-21 | 4 | 3 | 0,125 | 1 |
| Институт компьютерных наук | ИВТ-22 | 3 | 3 | 0,115 | 2 |
| Институт компьютерных наук | ИВТ-23 | 2 | 3 | 0,107 | 3 |
| Институт компьютерных наук | ПИ-21 | 4 | 2 | 0,091 | 4 |
| Институт компьютерных наук | ИБ-23 | 2 | 1 | 0,043 | 5 |
| Институт компьютерных наук | ПИ-22 | 3 | 1 | 0,040 | 6 |
| Институт прикладной математики | ПМ-21 | 4 | 3 | 0,150 | 1 |
| Институт прикладной математики | ПМ-22 | 3 | 1 | 0,048 | 2 |
| Механико-технологический институт | МТ-21 | 4 | 1 | 0,056 | 1 |
| Механико-технологический институт | МТ-22 | 3 | 1 | 0,053 | 2 |

## 3.5 Выборка 4. Ближайший контроль

Коррелированный `LATERAL` берёт для группы один экзамен с минимальной датой строго позже 15 января 2026 и подставляет название дисциплины по `SubjectId`.

```
SELECT g."GroupCode" AS "Группа",
       nxt."SubjectName" AS "БлижайшаяДисциплина",
       nxt."ExamDate" AS "Дата",
       nxt."TeacherFullName" AS "Преподаватель",
       nxt."AssessmentType" AS "Форма"
FROM "StudentGroups" g
JOIN LATERAL (
    SELECT s."SubjectName", e."ExamDate",
           e."TeacherFullName", e."AssessmentType"
    FROM "ExamSessions" e
    JOIN "Subjects" s ON s."Id" = e."SubjectId"
    WHERE e."StudentGroupId" = g."Id"
      AND e."ExamDate" > DATE '2026-01-15'
    ORDER BY e."ExamDate"
    LIMIT 1
) nxt ON TRUE
ORDER BY nxt."ExamDate", g."GroupCode";
```

Таблица 3: Вывод выборки 4, 11 строк

| Группа | Дата | Дисциплина | Форма | Преподаватель |
|---|---|---|---|---|
| ПИ-21 | 16.01 | Базы данных | Экзамен | Орлов И. С. |
| ИВТ-22 | 18.01 | Компьютерные сети | Экзамен | Орлов И. С. |
| ИВТ-21 | 19.01 | Операционные системы | Экзамен | Кузнецов П. А. |
| ПИ-22 | 21.01 | Операционные системы | Диф. зачёт | Кузнецов П. А. |
| ЭК-21 | 22.01 | Экономика предприятия | Экзамен | Белова М. О. |
| ПМ-21 | 26.01 | Алгоритмы и структуры данных | Экзамен | Морозова Е. В. |
| ЭК-22 | 27.01 | Бухгалтерский учёт | Диф. зачёт | Белова М. О. |
| МТ-21 | 29.01 | Технология машиностроения | Экзамен | Ершов С. П. |
| МТ-22 | 30.01 | Технология машиностроения | Экзамен | Ершов С. П. |
| ИВТ-23 | 05.06 | Программирование | Диф. зачёт | Морозова Е. В. |
| ИБ-23 | 12.06 | Информационная безопасность | Экзамен | Новикова О. Д. |

Одиннадцать строк. ПМ-22 в результат не попадает: её экзамен по дискретной математике стоит ровно 15 января, а условие строгое.

## 3.6 Изменение 1. Перевод четвёртого курса

Группы 4 курса института компьютерных наук переводятся на следующий курс. Численность уменьшается на двух отчисленных и не опускается ниже нуля.

```
UPDATE "StudentGroups"
SET "CourseYear"   = "CourseYear" + 1,
    "StudentCount" = GREATEST("StudentCount" - 2, 0)
WHERE "CourseYear" = 4
  AND "Faculty" = 'Институт␣компьютерных␣наук'
RETURNING "GroupCode", "CourseYear", "StudentCount";
```

До изменения условию отвечали ИВТ-21 (4 курс, 24) и ПИ-21 (4 курс, 22). Вывод оператора:

```
UPDATE 2
 GroupCode | CourseYear | StudentCount
```

```
-----------+------------+-------------
 ИВТ-21    |          5 |          22
 ПИ-21     |          5 |          20
```

Повторный отбор групп института оставляет остальные четыре строки прежними: ИВТ-22 (3 курс, 26), ПИ-22 (3 курс, 25), ИБ-23 (2 курс, 23), ИВТ-23 (2 курс, 28).

## 3.7 Изменение 2. Дифференцированный зачёт

Зачёты Соколовой по кафедре вычислительных систем становятся дифференцированными.

```
UPDATE "ExamSessions" e
SET "AssessmentType" = 'Дифференцированный зачёт'
FROM "Subjects" s
WHERE e."SubjectId" = s."Id"
  AND s."Department" = 'Кафедра вычислительных систем'
  AND e."TeacherFullName" = 'Соколова Анна Петровна'
  AND e."AssessmentType" = 'Зачёт'
RETURNING s."SubjectName", e."ExamDate", e."AssessmentType";
```

До изменения условию отвечала одна строка: ИВТ-23, базы данных, 10 июня 2026, зачёт. Вывод оператора:

```
UPDATE 1
 SubjectName |  ExamDate  |      AssessmentType
-------------+------------+--------------------------
 Базы данных | 2026-06-10 | Дифференцированный зачёт
```

Январские экзамены Соколовой не затронуты: у ИВТ-21 (12 января) и ИВТ-22 (14 января) форма «Экзамен», а не «Зачёт».

## 3.8 Вставка 1. Новая дисциплина и зачёт

В перечень добавляется дисциплина «Криптографические методы» и первый зачёт по ней для группы ИБ-23.

```
INSERT INTO "Subjects" ("SubjectName", "Department", "TotalHours")
VALUES ('Криптографические методы', 'Кафедра защиты информации', 72);

INSERT INTO "ExamSessions"
    ("StudentGroupId", "SubjectId", "ExamDate",
     "TeacherFullName", "AssessmentType")
SELECT g."Id", s."Id", DATE '2026-06-18',
       'Новикова Ольга Дмитриевна', 'Зачёт'
FROM "StudentGroups" g
JOIN "Subjects" s ON s."SubjectName" = 'Криптографические методы'
WHERE g."GroupCode" = 'ИБ-23';
```

Оба оператора вернули `INSERT 0 1`. Проверочная выборка по названию дисциплины дала одну строку:

```
 GroupCode | SubjectName              | Department
-----------+--------------------------+--------------------------
 ИБ-23     | Криптографические методы | Кафедра защиты информации

 TotalHours | ExamDate   | TeacherFullName           | AssessmentType
------------+------------+---------------------------+----------------
         72 | 2026-06-18 | Новикова Ольга Дмитриевна | Зачёт
```

## 3.9 Вставка 2. Пересдача баз данных

Январский экзамен по базам данных назначается повторно через 30 дней. Дата берётся из уже существующей строки.

```
INSERT INTO "ExamSessions"
    ("StudentGroupId", "SubjectId", "ExamDate",
     "TeacherFullName", "AssessmentType")
SELECT e."StudentGroupId", e."SubjectId", e."ExamDate" + 30,
       e."TeacherFullName", 'Экзамен'
FROM "ExamSessions" e
JOIN "Subjects" s ON s."Id" = e."SubjectId"
WHERE s."SubjectName" = 'Базы␣данных'
  AND e."AssessmentType" = 'Экзамен'
  AND e."ExamDate" < DATE '2026-02-01';
```

До вставки по базам данных было четыре строки: три январских экзамена и июньский дифференцированный зачёт ИВТ-23. Оператор вернул `INSERT 0 3`. Повторный отбор дал семь строк:

```
 GroupCode | ExamDate   | TeacherFullName         | AssessmentType
-----------+------------+-------------------------+----------------
 ИВТ-21    | 2026-01-12 | Соколова Анна Петровна | Экзамен
 ИВТ-21    | 2026-02-11 | Соколова Анна Петровна | Экзамен
 ИВТ-22    | 2026-01-14 | Соколова Анна Петровна | Экзамен
 ИВТ-22    | 2026-02-13 | Соколова Анна Петровна | Экзамен
 ИВТ-23    | 2026-06-10 | Соколова Анна Петровна | Диф. зачёт
 ПИ-21     | 2026-01-16 | Орлов Игорь Сергеевич  | Экзамен
 ПИ-21     | 2026-02-15 | Орлов Игорь Сергеевич  | Экзамен
```

Июньский зачёт ИВТ-23 не продублирован: фильтр требует форму «Экзамен» и дату раньше 1 февраля.

## 3.10 Удаление 1. Снятие пересдачи

Пересдача снимается, если по той же паре группа–дисциплина уже есть более ранний экзамен. Проверка идёт коррелированным `EXISTS`.

```
DELETE FROM "ExamSessions" e
USING "Subjects" s
WHERE e."SubjectId" = s."Id"
  AND s."SubjectName" = 'Базы␣данных'
  AND e."ExamDate" >= DATE '2026-02-01'
  AND EXISTS (
      SELECT 1
      FROM "ExamSessions" earlier
      WHERE earlier."StudentGroupId" = e."StudentGroupId"
        AND earlier."SubjectId" = e."SubjectId"
        AND earlier."ExamDate" < e."ExamDate"
  );
```

Оператор вернул `DELETE 3`: сняты 11, 13 и 15 февраля. Повторный отбор снова даёт четыре исходные строки:

```
 GroupCode | ExamDate   | TeacherFullName         | AssessmentType
-----------+------------+-------------------------+--------------------------
 ИВТ-21    | 2026-01-12 | Соколова Анна Петровна | Экзамен
 ИВТ-22    | 2026-01-14 | Соколова Анна Петровна | Экзамен
 ИВТ-23    | 2026-06-10 | Соколова Анна Петровна | Дифференцированный зачёт
 ПИ-21     | 2026-01-16 | Орлов Игорь Сергеевич  | Экзамен
```

### 3.11 Удаление 2. Дисциплины вне сессии

Удаляются дисциплины, которым не соответствует ни одна строка экзамена.

```
DELETE FROM "Subjects" s
WHERE NOT EXISTS (
    SELECT 1 FROM "ExamSessions" e WHERE e."SubjectId" = s."Id"
)
RETURNING s."SubjectName", s."Department";
```

До удаления таким был единственный предмет. Вывод оператора:

```
DELETE 1
 SubjectName | Department
-------------+--------------------------
 Философия   | Кафедра гуманитарных наук
```

Криптографические методы сохранены: на них ссылается зачёт ИБ-23. После удаления в таблице 13 дисциплин, и каждая встречается в сессии.

## 4 Задание 1.4. Контроль целостности

### 4.1 Постановка

Проверить потерянное обновление, грязное чтение, неповторяющееся чтение, фантом и аномалию сериализации на уровнях SQL/92. Написать два триггера: один поправляет данные, второй откатывает транзакцию.

Опыты идут по `StudentGroups.StudentCount`. Исходно ИВТ-21 — 24, ИВТ-22 — 26. В PostgreSQL READ UNCOMMITTED работает как READ COMMITTED, поэтому грязного чтения нет ни на одном уровне.

### 4.2 Сводка

Таблица 4: Аномалии в PostgreSQL 16

| Аномалия | RC (и RU) | RR | Serializable |
|---|---|---|---|
| Потерянное обновление | нет | нет | нет |
| Грязное чтение | нет | нет | нет |
| Неповторяющееся чтение | да | нет | нет |
| Фантом | да | нет | нет |
| Сериализация | — | да | откат |

### 4.3 Потерянное обновление

Проверяется инкрементами, не присваиванием константы. A прибавляет 1, B прибавляет 10. Без аномалии итог равен $24 + 1 + 10 = 35$.

Таблица 5: Ход сеансов, READ COMMITTED

| Транзакция A | Транзакция B |
|---|---|
| BEGIN, READ COMMITTED | |
| | BEGIN, READ COMMITTED |
| UPDATE +1, видит 25 | |
| | UPDATE +10, ждёт блокировку |
| COMMIT | |
| | дожидается, пишет 35, COMMIT |

Итог — 35, оба инкремента живы. На REPEATABLE READ строка блокируется так же.

## 4.4 Грязное и неповторяющееся чтение

A прибавляет 100 и не фиксирует транзакцию. B читает ИВТ-21 и получает 24, а не 124. После отката A чужое значение так и не стало видимым.

Неповторяющееся чтение: A читает 24, B пишет 40 и фиксируется, A читает снова. READ COMMITTED возвращает 40, REPEATABLE READ возвращает 24.

## 4.5 Фантом

A дважды считает группы первого курса. Между подсчётами B вставляет PHANTOM-99 с курсом 1. READ COMMITTED меняет счётчик с 0 на 1, REPEATABLE READ оба раза даёт 0.

## 4.6 Сериализация

A пишет в ИВТ-21 значение ИВТ-22. B пишет в ИВТ-22 значение ИВТ-21 из своего снимка. Последовательно A, затем B, даёт (26, 26). Параллельно на REPEATABLE READ получается (26, 24): такого состояния нет ни в одном последовательном порядке. На SERIALIZABLE вторая транзакция получает ошибку `could not serialize access` и откатывается, остаётся (26, 26).

## 4.7 Триггеры

`trg_limit_student_drop` срабатывает до обновления численности. Падение больше чем на 5 человек срезается, транзакция остаётся жива. МТ-22 имела 19 человек, запрос ставил 1, в таблице осталось 14.

`trg_check_exam_date` запрещает экзамен в выходной и второй контроль группы в тот же день. 20 июня 2026 — суббота, вставка экзамена МТ-21 отклонена. Вставка компьютерных сетей для ИВТ-21 на 12 января отклонена: в этот день у группы уже стоят базы данных. Оба отказа откатывают оператор.

## 4.8 Выводы по заданию 1.4

В PostgreSQL READ UNCOMMITTED работает как READ COMMITTED, поэтому грязного чтения нет ни на одном уровне. Потерянное обновление проверяется инкрементами, а не присваиванием константы. Аномалия сериализации получается перекрёстным обновлением двух строк. На SERIALIZABLE вторая транзакция откатывается, на REPEATABLE READ аномалия остаётся.

# 5 Задание 2.1. Управление доступом

## 5.1 Постановка

Создать пользователя `test`, раздать разные права на разные таблицы, сделать однотабличное и многотабличное представления, разрешить вставку в многотабличное представление и завести роль с UPDATE на отдельные столбцы. Права показать командой `\dp`.

## 5.2 Права и представления

`test` получает LOGIN, `CONNECT` на базу и `USAGE` на схему. На `StudentGroups` выданы SELECT, INSERT и UPDATE. На `Subjects` — чтение всех столбцов и UPDATE только `TotalHours`. На `ExamSessions` — только SELECT.

`v_exam_card` — представление экзаменов без зачётов. Оно соединяет `ExamSessions` с группой и дисциплиной, чтобы в карточке был код группы и название, а не суррогатные ключи. `test` читает из него 16 строк. `v_group_load` соединяет группу с числом контролей. Вставку обрабатывает триггер `INSTEAD OF INSERT`: пишется строка группы, вычисляемый столбец игнорируется, `Id` группа получает сама. От имени `test` так создана группа ТЕСТ-01, 1 курс, 17 человек.

Роль `exam_editor` выдана пользователю `test`. Через представление `v_group_headcount` она меняет только `StudentCount` и только у института компьютерных наук: у представления стоят `security_barrier` и `check_option`. Материализованное `mv_faculty_stats` хранит готовый агрегат по факультетам и не пересчитывает соединение на каждый просмотр.

## 5.3 Проверка от имени test

SELECT по группе проходит. UPDATE часов баз данных проходит: 144 становится 150. UPDATE кафедры отклоняется, `permission denied for table Subjects`. INSERT в `ExamSessions` отклоняется так же. UPDATE численности ИВТ-23 через представление роли проходит, значение становится 29. Такой же UPDATE для МТ-21 меняет 0 строк: чужой факультет не проходит `CHECK OPTION`.

Команда `\dp exam.*` показывает флаги: `r` — чтение, `a` — вставка, `w` — обновление, `d` — удаление, `D` — очистка, `x` — ссылка, `t` — триггер. У `ExamSessions` для `test` стоит `r`, у `StudentGroups` — `arw`. В столбцовых привилегиях `TotalHours` имеет `rw`, остальные столбцы `Subjects` — `r`.

## 5.4 Выводы по заданию 2.1

Столбцовые права в выводе `\dp` выводятся отдельной колонкой, рядом с табличными флагами их легко пропустить. `CHECK OPTION` не заменяет привилегию: МТ-21 не обновилась, потому что строка не видна представлению, а не потому что нет права. Вставка в соединение без `INSTEAD OF` невозможна: строку нельзя однозначно разложить по таблицам. На Windows перед `\dp` в `psql` нужна команда `\! chcp 1251`, иначе кириллица в именах распадается.

# 6 Задание 2.2. Функции и PL/pgSQL

## 6.1 Постановка

Написать три-четыре функции, которые упрощают работу с данными. В коде должны быть переменные, ветвления, циклы, курсор и обработка исключений. Отдельно показать,

чем функция удобнее представления.

## 6.2 Назначение экзамена

`exam.schedule_exam` проверяет, что группа и дисциплина существуют, и берёт их `Id`. Форма контроля должна входить в допустимый список. Повторная вставка той же тройки группа–дисциплина–дата перехватывается секцией `EXCEPTION` по `unique_violation` и заменяется понятным текстом. Уникальность держит отдельное ограничение, а не первичный ключ. Успешный вызов поставил зачёт МТ-21 по базам данных на 22 июня 2026. Повторный вызов вернул «контроль уже назначен». Код — `sql/07_functions.sql`.

## 6.3 Ведомость преподавателя

`exam.teacher_sheet` открывает курсор по экзаменам преподавателя, в цикле `LOOP` забирает строку и ветвлением `IF/ELSIF` переводит форму контроля в короткий код. Для Соколовой функция вернула три строки:

```
1. 2026-01-12  ИВТ-21  ЭКЗ  Базы данных
2. 2026-01-14  ИВТ-22  ЭКЗ  Базы данных
3. 2026-06-10  ИВТ-23  ЗАЧ  Базы данных
```

## 6.4 Перенос сессии группы

`exam.postpone_group` в цикле `FOR` по выборке сдвигает даты группы на заданное число дней и внутренним циклом перепрыгивает субботу и воскресенье. Неположительный сдвиг отвергается исключением. Для МТ-22 сдвиг на 3 дня перенёс экзамен с 30 января на 2 февраля: 1 февраля 2026 — воскресенье, функция ушла на понедельник. Возвращено число сдвинутых строк, оно равно 1.

## 6.5 Сводка факультета

`exam.faculty_report` возвращает таблицу из четырёх столбцов. Переменные держат число групп, студентов и контролей, ветвление собирает комментарий. Для института компьютерных наук: 6 групп, 148 студентов, 13 контролей, «нагрузка выше двух контролей на группу». Неизвестный факультет поднимает исключение, а не пустую таблицу.

## 6.6 Функция против представления

Представление — сохранённый запрос без параметров и без права прервать операцию. Функция принимает факультет и преподавателя аргументами, проверяет вход до изменения данных и возвращает либо таблицу, либо текст ведомости. Отказ с текстом на русском сообщает причину, а не сырой код нарушения ключа. Карточка экзамена из задания 2.1 остаётся представлением: ей не нужны параметры и проверка входа.

## 6.7 Выводы по заданию 2.2

Ведомость можно собрать одним запросом. Курсор оставлен, чтобы показать построчную обработку: `FETCH`, `EXIT WHEN NOT FOUND` и ветвление по форме контроля. Сдвиг дат идёт циклом, потому что правило выходного дня в `WHILE` записывается прямо. Секция `EXCEPTION` ловит конкретный `SQLSTATE`. Перехват `OTHERS` скрыл бы и сбой схемы, и настоящий отказ ограничения.

# Заключение

Собрана база учёта сессии из двух стержневых сущностей и одной ассоциативной. Идентификаторы сущностей — `UUID`, связи идут по ним, а код группы и название дисциплины остаются уникальными естественными ключами. База заполнена и проверена запросами с соединением, CTE, окном и коррелированным подзапросом. В PostgreSQL грязного чтения нет: READ UNCOMMITTED совпадает с READ COMMITTED. Потерянное обновление закрыто блокировкой строки. Аномалия сериализации возникает на REPEATABLE READ и откатывается на SERIALIZABLE. Триггеры проверяют условия, которые не выражаются столбцовым `CHECK`. Роли и представления отрезают пользователю `test` чужие столбцы и чужие таблицы. Функции принимают параметры, проверяют вход и возвращают текст ошибки.